BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería en Ciencias de la Computación

1er Exámen Parcial

Reporte Técnico

Asignatura:	Minería de Datos (ICCS-257 | NRC: 25467)
Profesor:	Dr. Iván Olmos Pineda
Alumno:	Gael Askary Razo Montañez
Matrícula:	202352659
Periodo Académico:	Otoño 2026
Fecha de Entrega:	3 de noviembre de 2026

Puebla de Zaragoza, Pue.

Índice

1. Introducción **2**
1.1. Naturaleza Estocástica de la Etiqueta y Techo de Predictibilidad 2

2. Desarrollo **3**
2.1. Pregunta 1: Preprocesamiento y Limpieza de Datos 3
2.2. Pregunta 2: Muestreo Estadístico y Particionado Estratificado 5
2.3. Pregunta 3: Diseño de Validación Cruzada Estratificada 6
2.4. Pregunta 4: Entrenamiento del Perceptrón Multicapa y Búsqueda de Hiperparámetros 7
2.5. Pregunta 5: Argumentación de Métricas de Evaluación 9
2.6. Pregunta 6: Mejor Modelo sin Sobreajuste y Variación Estadística 11
2.7. Pregunta 7: Video Explicativo, Ventajas y Áreas de Mejora 13

3. Gráficas y Resultados Experimentales **13**

4. Conclusiones **19**

5. Referencias Bibliográficas **20**

1 Introducción

El presente proyecto aborda el problema de clasificación supervisada multiclase sobre una base de datos de comercio electrónico compuesta por 1,000,000 de registros y 31 atributos. Cada fila resume la actividad transaccional, navegacional, promocional y de posventa de un cliente durante los 365 días previos a la fecha de corte (30 de junio de 2026), e incluye la variable objetivo `clase_cliente`, la cual clasifica el comportamiento del consumidor durante los **90 días posteriores al corte** (1 de julio al 28 de septiembre de 2026).

De acuerdo con la especificación técnica del dominio comercial, la etiqueta futura se asigna mediante una estructura jerárquica de cinco prioridades excluyentes basada en las compras futuras (N), compras futuras con descuento (D) y gasto futuro (G) dentro de la ventana prospectiva de 90 días:

1. **Prioridad 1** — `Inactivo`**:** clientes sin compras en los 90 días siguientes ($N = 0$).
2. **Prioridad 2** — `Alto_valor`**:** clientes activos con alto desembolso ($G \geq$ \$14,000 MXN o $G \geq$ \$10,500 MXN acompañado de al menos $N \geq 7$ compras).
3. **Prioridad 3** — `Promocional`**:** clientes activos que no cumplen alto valor pero cuya proporción de compras futuras con descuento verifica $D/N \geq 0.60$ (con $D \geq 2$), o bien $D \geq 4$ junto con un descuento histórico promedio $\geq 20\,\%$.
4. **Prioridad 4** — `Frecuente`**:** clientes activos con recurrencia sostenida ($N \geq 5$ compras futuras) que no dependen de promociones ($D/N < 0.60$) y mantienen un gasto moderado ($G \geq$ \$3,200 MXN).
5. **Prioridad 5** — `Ocasional`**:** clientes con actividad moderada o esporádica ($N \geq 1$) que no satisfacen las condiciones de los segmentos anteriores.

1.1 Naturaleza Estocástica de la Etiqueta y Techo de Predictibilidad

Un aspecto metodológico fundamental de este problema es que las cantidades prospectivas (N, D, G) de los 90 días posteriores al corte **no forman parte de las variables de entrada**; el clasificador únicamente observa el resumen histórico de 365 días. Dado que la ocurrencia exacta de compras en un trimestre futuro contiene variabilidad aleatoria (por ejemplo, clientes con historiales similares que realizan 0 o 1 compra en el trimestre siguiente, cruzando la frontera entre `Inactivo` y `Ocasional`), existe una tasa de error irreducible de Bayes ($\varepsilon_{\text{Bayes}} > 0$) distinta de cero. En consecuencia, el diseño del sistema se enfoca en:

- Rescatar de forma determinista todos los registros reparables durante el preprocesamiento sin introducir sesgos de imputación artificial.
- Preservar rigurosamente la estructura poblacional mediante muestreo de Cochran con corrección por población finita y particionado estratificado sin fuga de información (*data leakage*).
- Optimizar un Perceptrón Multicapa (`MLPClassifier`) explorando sistemáticamente arquitecturas, funciones de activación, regularización L_2, algoritmos de optimización (`adam` y `sgd` con `momentum` y Nesterov) y ponderación por clase (γ), evaluando su estabilidad estadística con validación cruzada y remuestreo *bootstrap*.

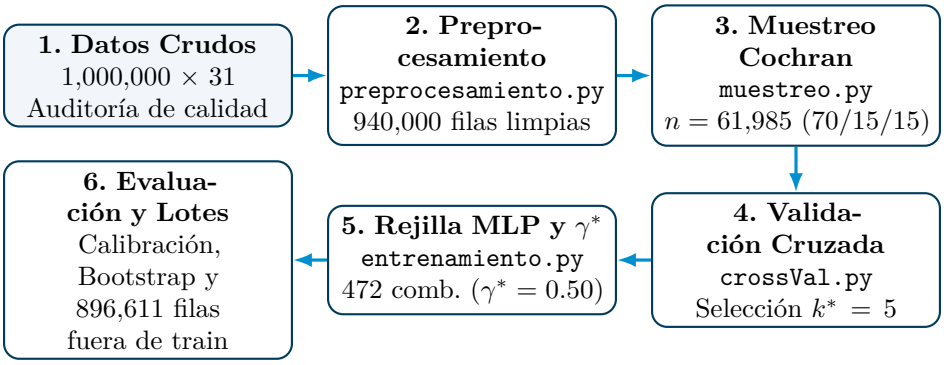


Figura 1: Arquitectura general del flujo de minería de datos libre de fuga de información.

2 Desarrollo

2.1 Pregunta 1: Preprocesamiento y Limpieza de Datos

Pregunta 1 — Preprocesamiento (`preprocesamiento.py`)

¿Qué técnicas de preprocesamiento se aplicaron a la base de datos original para corregir problemas de calidad (valores nulos, registros duplicados, valores fuera de rango, inconsistencias en variables categóricas, formato numérico y relaciones entre variables), cuántos registros se corrigieron o eliminaron por cada criterio y cuál es la justificación técnica de cada decisión?

Respuesta

El módulo `preprocesamiento.py` implementa un flujo de auditoría y limpieza determinista en dos etapas claramente diferenciadas: (1) **normalización y rescate de registros reparables**, y (2) **filtrado por reglas de integridad de dominio** con atribución por causa excluyente. Esta separación evita descartar observaciones cuya aparente invalidez proviene únicamente de variaciones de formato léxico o monetario.

1. Diagnóstico estructural del archivo crudo (1,000,000 de filas). La inspección del archivo `clientes_comercio_1000000_alumnos.csv` identificó exactamente cuatro bloques disjuntos de 30,000 registros cada uno con anomalías inyectadas, además de 0 duplicados exactos, 0 identificadores `id_cliente` repetidos y 0 violaciones de precedencia lógica:

- **Bloque A (Formato monetario y variantes léxicas categóricas — 30,000 filas reparables):**
  - 7,562 registros en `ticket_promedio_mxn` contenían símbolos monetarios, separadores de miles o sufijos de divisa (por ejemplo, `"$1,245.50 MXN"`). La función `clean_numeric_text` elimina los caracteres `$`, comas, espacios y el sufijo `MXN` mediante expresiones regulares antes de la conversión numérica, rescatando el 100 % de estos valores sin pérdida.
  - 22,438 registros presentaban variantes de escritura en tres variables categóricas: `region` (7,517 casos como `"NORTE"`, `"sur "`), `canal_preferido` (7,469 casos como `"web"`, `.ᵃPP"`) y `metodo_pago_preferido` (7,452 casos como `Cartera digital"` con espacio en lugar de guion bajo, o `"tarjeta"`). La función `normalize_categorical_series` aplica descomposición Unicode (`NFD`), eliminación de marcas diacríticas, conversión a minúsculas y unificación de espacios por guion bajo para mapear cada variante a su etiqueta canónica oficial del diccionario de datos (`DOMAINS`), rescatando íntegramente los 22,438 registros ($7{,}562 + 22{,}438 = 30{,}000$).

- **Bloque B (Inconsistencia aritmética en gasto anual — 30,000 filas reparables):** El documento técnico define la identidad exacta:

$$\texttt{gasto_total_365d_mxn} = \text{round}(\texttt{compras_365d} \times \texttt{ticket_promedio_mxn}, 2)\,. \tag{1}$$

  Se detectaron 30,000 registros donde el gasto almacenado discrepaba de dicho producto en más de \$0.05 MXN (1,034 casos con `compras_365d` = 0 pero gasto positivo, y 28,966 casos con `compras_365d` > 0 e importe corrompido). Dado que ambos factores (`compras_365d` y `ticket_promedio_mxn`) estaban presentes y dentro de rango, se recalculó de manera exacta `gasto_total_365d_mxn` aplicando la ecuación (1), conservando las 30,000 filas.
- **Bloque C (Valores nulos genuinos — 30,000 filas eliminadas):** Una vez rescatados los formatos monetarios, permanecieron exactamente 30,000 registros con valores faltantes (`NaN`) distribuidos en siete atributos: `edad` (4,297), `duracion_sesion_min` (4,349), `metodo_pago_preferido` (4,331), `ticket_promedio_mxn` (4,287), `paginas_por_sesion` (4,264), `satisfaccion_promedio` (4,238) y `region` (4,234). Estos valores no pueden deducirse de forma determinista del resto de columnas; imputarlos con la media o moda antes del particionado distorsionaría la varianza marginal y crearía combinaciones sintéticas. Representando apenas el 3.0 % de la base, se eliminaron.
- **Bloque D (Valores fuera de rango físico o de diccionario — 30,000 filas eliminadas):** Se identificaron 30,000 registros que violaban las cotas del diccionario de datos: `edad` $\notin [18, 80]$ (10,026 casos, incluyendo edades negativas o superiores a 120 años), `paginas_por_sesion` $\notin [1, 25]$ (9,935 casos) y `descuento_promedio_pct` $\notin [0, 65]$ (10,039 casos). Al tratarse de errores de captura no recuperables determinísticamente, fueron descartados.

Tabla 1: Contabilidad exacta del preprocesamiento en `preprocesamiento.py` (fuente: `auditoria_preprocesamiento.json`).

Concepto / Regla de Auditoría	**Filas**	**% Base**	**Acción Técnica y Justificación**
Registros de entrada (`input_rows`)	1,000,000	100.00 %	Lectura tipada del archivo crudo original.
Registros corregidos determinísticamente y conservados (60,000 filas):			
Formato monetario en `ticket_promedio_mxn`	7,562	0.76 %	Limpieza de `$`, comas y sufijo `MXN`.
Normalización léxica en `region`	7,517	0.75 %	Mapeo canónico sin acentos/mayúsculas.
Normalización léxica en `canal_preferido`	7,469	0.75 %	Unificación a `{App, Web, Tienda}`.
Normalización léxica en `metodo_pago_preferido`	7,452	0.75 %	Unificación de `Çartera digital"` → `Cartera_digital`.
Recálculo exacto de `gasto_total_365d_mxn`	30,000	3.00 %	round(compras × ticket, 2) (1,034 con compras = 0 y 28,966 con compras > 0).
Registros eliminados por causa excluyente (60,000 filas):			
1. Valores nulos irrecuperables (`nulos`)	30,000	3.00 %	Nulos reales en 7 atributos sin regla determinista.
2. Fuera de rango de dominio (`fuera_de_rango`)	30,000	3.00 %	Violaciones en `edad`, `paginas_por_sesion` o `descuento_promedio_pct`.
3. Violaciones de dependencias lógicas	0	0.00 %	`compras_con_descuento` ≤ `compras`, `devoluciones` ≤ `compras`, `emails_abiertos` ≤ `emails_enviados`.
4. Duplicados exactos o de `id_cliente`	0	0.00 %	Unicidad verificada al 100 %.
Registros limpios conservados (`output_rows`)	**940,000**	**94.00 %**	Exportados en `datos/dataset_limpio.csv`.

2. Tratamiento de valores extremos (Outliers IQR) y correlaciones. Sobre los 940,000 registros válidos se cuantificaron los valores extremos mediante el criterio intercuartílico $[Q_1 -$

1.5 IQR, $Q_3 + 1.5\,\text{IQR}$], detectándose colas derechas pronunciadas en variables transaccionales como `gasto_total_365d_mxn` (57,806 observaciones extremas), `reclamos_365d` (51,464) y `puntos_lealtad` (33,110). **Estos registros no se eliminaron ni truncaron**, dado que corresponden al comportamiento real de los clientes del segmento `Alto_valor` (6.78 % de la población); suprimirlos cercenaría la clase más crítica para el negocio. Tras la limpieza, la correlación de Pearson entre `gasto_total_365d_mxn` y `compras_365d` es $r = 0.7558$, y con `ticket_promedio_mxn` es $r = 0.7318$.

2.2 Pregunta 2: Muestreo Estadístico y Particionado Estratificado

Pregunta 2 — Muestreo (`muestreo.py`)

¿Qué técnica de muestreo se utilizó para seleccionar un subconjunto representativo de la base limpia, cómo se determinó matemáticamente su tamaño y cómo se garantizó que la muestra y las particiones (`train`, `validation`, `test`) conservaran la distribución estadística original?

Respuesta

El script `muestreo.py` aplica **muestreo aleatorio estratificado con afijación proporcional**, dimensionado mediante la fórmula de Cochran para poblaciones finitas. Entrenar una búsqueda exhaustiva de hiperparámetros de redes neuronales sobre 940,000 registros incrementaría en más de un orden de magnitud el tiempo de cómputo por pliegue sin ganancia predictiva apreciable (como se demuestra empíricamente en la curva de tamaño de muestra de la Pregunta 6).

1. Cálculo del tamaño muestral de Cochran con corrección por población finita (FPC). Para una población limpia de tamaño $N = 940{,}000$, fijando un nivel de confianza del 99 % ($Z = 2.576$), varianza máxima conservadora ($p = q = 0.5$) y un margen de error máximo admisible $e = 0.005$ ($\pm 0.5\,\%$), el tamaño muestral inicial n_0 y el tamaño ajustado n son:

$$n_0 = \frac{Z^2 \cdot p(1-p)}{e^2} = \frac{(2.576)^2 \cdot 0.25}{(0.005)^2} = 66{,}357.76, \tag{2}$$

$$n_{\text{FPC}} = \left\lceil \frac{n_0}{1 + \dfrac{n_0 - 1}{N}} \right\rceil = \left\lceil \frac{66{,}357.76}{1 + \dfrac{66{,}356.76}{940{,}000}} \right\rceil = 61{,}983 \text{ registros.} \tag{3}$$

Al aplicar afijación proporcional con redondeo hacia arriba por estrato ($n_c = \lceil n_{\text{FPC}} \cdot (N_c/N) \rceil$) para no subrepresentar ninguna clase, se obtiene una muestra estratificada total de **61,985 observaciones**, la cual se divide de forma estratificada (con semilla fija `SEED` $= 42$) en:

- **Entrenamiento (`datos/train.csv` — 70 %):** 43,389 registros (usados para validación cruzada, selección de k^*, selección de γ^* y ajuste de la rejilla).
- **Validación (`datos/validation.csv` — 15 %):** 9,298 registros (usados para calibración probabilística post-hoc, ajuste de multiplicadores de decisión e importancia por permutación).
- **Prueba (`datos/test.csv` — 15 %):** 9,298 registros (reservados exclusivamente para la evaluación final fuera de muestra).

Tabla 2: Conservación exacta de proporciones por clase entre la población limpia y las particiones estratificadas (fuente: `distribucion_particiones.csv` y `calculo_muestra.json`).

Clase	Población (N)	Prop. (%)	Muestra Cochran	Train (70 %)	Val (15 %)	Test (15 %)	p-valor χ^2
`Ocasional`	368,300	39.1809 %	24,286	17,000	3,643	3,643	0.9984
`Inactivo`	305,513	32.5014 %	20,146	14,102	3,022	3,022	0.9999
`Frecuente`	116,485	12.3920 %	7,681	5,377	1,152	1,152	0.9981
`Promocional`	86,011	9.1501 %	5,672	3,970	851	851	0.9966
`Alto_valor`	63,691	6.7756 %	4,200	2,940	630	630	0.9985
Total	**940,000**	**100.0000 %**	**61,985**	**43,389**	**9,298**	**9,298**	—

2. Verificación estadística de representatividad. Las pruebas de bondad de ajuste χ^2 por clase entre la población limpia y la muestra estratificada arrojan p-valores entre 0.9966 y 0.9999 (Tabla 2), confirmando que las frecuencias relativas son estadísticamente indistinguibles de la población ($|p_{\text{muestra}} - p_{\text{población}}| < 0.0003\,\%$). Adicionalmente, se ejecutaron pruebas no paramétricas de Kolmogorov–Smirnov de dos muestras sobre las 23 variables numéricas continuas y discretas, obteniendo $p > 0.05$ en la totalidad de los atributos (desde $p = 0.0525$ en `emails_abiertos_365d` hasta $p = 0.9999$ en `clics_anuncios_365d`), lo que demuestra que no se indujo sesgo de selección en ninguna dimensión.

2.3 Pregunta 3: Diseño de Validación Cruzada Estratificada

Pregunta 3 — Validación Cruzada (`crossVal.py`)

¿Qué esquema de validación cruzada se implementó, qué valores de k se compararon experimentalmente y con base en qué criterios estadísticos y de soporte muestral se seleccionó el número óptimo de pliegues k^*?

Respuesta

El módulo `crossVal.py` implementa un esquema de **Validación Cruzada Estratificada de k pliegues** (`StratifiedKFold(n_splits=k, shuffle=True, random_state=42)`) exclusivamente sobre el conjunto de entrenamiento (`train.csv`, 43,389 filas). En cada iteración, tanto el transformador de ingeniería de variables (`DomainFeatureTransformer`) como el estandarizador (`StandardScaler`) y el codificador *one-hot* (`OneHotEncoder`) se ajustan únicamente sobre los $k-1$ pliegues de entrenamiento del fold activo, impidiendo cualquier contaminación hacia el pliegue de validación interna.

Regla de decisión basada en soporte de la clase minoritaria y estabilidad. En problemas multiclase desbalanceados, elegir un valor de k excesivamente grande fragmenta el soporte de validación de la clase minoritaria (`Alto_valor`, que cuenta con 2,940 instancias en `train.csv`), incrementando la varianza de las métricas por clase en cada pliegue y multiplicando el costo computacional de la búsqueda en rejilla. Por ello, se estableció la regla formal:

> *Seleccionar el mayor $k \in \{3, 5, 10\}$ tal que cada pliegue de validación contenga al menos* **500** *observaciones de la clase minoritaria `Alto_valor`; en caso de empate en factibilidad, desempatar por menor desviación estándar de `F1-Macro`.*

Tabla 3: Evaluación experimental de $k \in \{3, 5, 10\}$ con el pipeline MLP base sobre `train.csv` (fuente: `reporte_crossval.csv` y `reporte_crossval.json`).

k	Alto_valor/fold	≥ 500	F1-Macro	DE	SE	Bal. Acc.	Decisión
3	980	Sí	0.2740	0.0054	0.0031	0.2701	Factible (menor % de datos de ajuste: 66.7 %)
5	**588**	**Sí**	**0.2675**	**0.0143**	**0.0064**	**0.2661**	**Seleccionado ($k^* = 5$, 80 % datos por fit)**
10	294	No	0.2732	0.0098	0.0031	0.2705	Descartado por soporte insuficiente ($294 < 500$)

Como muestra la Tabla 3, para $k = 10$ el soporte de `Alto_valor` cae a 294 registros por pliegue (incumpliendo el umbral de 500 casos y duplicando el número de ajustes de la rejilla de 2,360 a 4,720). Entre los valores factibles ($k = 3$ con 980 casos y $k = 5$ con 588 casos), la regla selecciona $\mathbf{k^* = 5}$, ya que preserva $588 \geq 500$ clientes de `Alto_valor` en cada pliegue de evaluación y destina el 80 % del conjunto de entrenamiento (34,711 registros) al ajuste de los pesos sinápticos en cada partición.

2.4 Pregunta 4: Entrenamiento del Perceptrón Multicapa y Búsqueda de Hiperparámetros

Pregunta 4 — Entrenamiento y Optimización del MLP (`entrenamiento.py`)

¿Cómo se estructuró el entrenamiento del Perceptrón Multicapa (`MLPClassifier`), qué espacio de hiperparámetros (# de neuronas, # de capas ocultas, # de épocas, tasa de aprendizaje, `momentum`, función de activación, regularización y tamaño de lote) se exploró y cuáles fueron los parámetros óptimos encontrados?

Respuesta

1. Formulación matemática y ponderación por clase (γ). Sea $\mathbf{x} \in \mathbb{R}^d$ el vector de entrada preprocesado ($d = 23$ variables numéricas originales +8 variables de dominio +23 indicadores *one-hot* = 54 características). Para una red de L capas ocultas, la activación de la capa $\ell \in \{1, \ldots, L\}$ se define recursivamente como $\mathbf{h}^{(\ell)} = \phi\left(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right)$ con $\mathbf{h}^{(0)} = \mathbf{x}$ y función no lineal $\phi \in \{\text{ReLU}, \tanh\}$. La capa de salida proyecta sobre las $K = 5$ clases mediante la función *softmax*:

$$\hat{y}_{ic} = P(y_i = c \mid \mathbf{x}_i) = \frac{\exp\left(\mathbf{w}_c^\top \mathbf{h}_i^{(L)} + b_c\right)}{\sum_{k=1}^{K} \exp\left(\mathbf{w}_k^\top \mathbf{h}_i^{(L)} + b_k\right)}. \tag{4}$$

Para evitar que las clases mayoritarias (`Ocasional` con 39.18 % e `Inactivo` con 32.50 %) dominen el gradiente en detrimento de las clases minoritarias (`Alto_valor`, `Promocional` y `Frecuente`), cada observación recibe un peso muestral amortiguado parametrizado por el exponente $\gamma \in [0, 1]$:

$$w_c(\gamma) = \left(\frac{N_{\text{train}}}{K \cdot N_c}\right)^{\gamma}, \qquad \mathcal{L}_\gamma(\Theta) = -\frac{1}{\sum_{i=1}^{n} w_{y_i}(\gamma)} \sum_{i=1}^{n} w_{y_i}(\gamma) \sum_{c=1}^{K} y_{ic} \log \hat{y}_{ic} + \frac{\alpha}{2n} \sum_{\ell=1}^{L+1} \left\|\mathbf{W}^{(\ell)}\right\|_F^2. \tag{5}$$

En `scikit-learn 1.9.1`, `MLPClassifier.fit` soporta nativamente el argumento `sample_weight`, el cual se enruta de forma limpia dentro del `Pipeline` mediante `model__sample_weight`.

2. Elección empírica de γ^* mediante la regla de un error estándar (1-SE). Antes de ejecutar la rejilla principal, la función `sweep_gamma` evaluó mediante validación cruzada de 5 pliegues sobre `train.csv` la cuadrícula $\gamma \in \{0.00, 0.25, 0.50, 0.75\}$ aplicando la regla de parsimonia de un error

estándar (1-SE): elegir el menor γ cuyo `F1-Macro` medio se ubique a no más de un error estándar del máximo observado.

Tabla 4: Barrido del exponente de balanceo γ en validación cruzada de 5 pliegues sobre `train.csv` (fuente: `barrido_gamma.csv`).

γ	F1-Macro	DE (F1)	SE	Bal. Acc.	Accuracy	Criterio 1-SE (≥ 0.3247)
0.00	0.2782	0.0099	0.0044	0.2757	0.4227	No cumple (sesgo hacia mayoritarias)
0.25	0.3204	0.0085	0.0038	0.3143	0.4158	No cumple ($0.3204 < 0.3247$)
0.50	**0.3249**	**0.0038**	**0.0017**	**0.3372**	**0.3914**	**Seleccionado ($\gamma^* = 0.50$, menor varianza)**
0.75	0.3261	0.0032	0.0014	0.3529	0.3638	Máximo nominal (umbral $= 0.3261 - 0.0014 = 0.3247$)

El máximo nominal ocurre en $\gamma = 0.75$ (F1-Macro $= 0.3261 \pm 0.0014$), fijando el umbral 1-SE en $0.3261 - 0.0014 = 0.3247$. El valor $\boldsymbol{\gamma^* = 0.50}$ supera dicho umbral ($0.3249 \geq 0.3247$) al tiempo que conserva 2.76 puntos porcentuales adicionales de exactitud global (0.3914 frente a 0.3638), evitando sobreponderar agresivamente el ruido en la frontera de las clases minoritarias.

3. Ingeniería de variables del dominio comercial e importancia por permutación. El transformador sin fuga `DomainFeatureTransformer` incorpora 8 razones numéricas diseñadas específicamente a partir de las reglas de negocio (usando denominadores protegidos $\max(\cdot, 1)$): `ratio_descuento`, `frecuencia_anualizada`, `intensidad_recencia`, `unidades_por_compra`, `tasa_abandono_carrito`, `tasa_apertura_emails`, `tasa_devoluciones` e `indice_valor_cliente`. Al evaluar la importancia por permutación (`permutation_importance` con 10 repeticiones sobre el conjunto completo de validación, permutando cada una de las 29 variables originales a la entrada del pipeline completo para propagar su efecto hacia las variables derivadas), se confirma que la señal predictiva está concentrada en tres dimensiones clave:

Tabla 5: Top 8 variables originales según importancia por permutación sobre `validation.csv` ($n_{\text{repeats}} = 10$, fuente: `importancia_permutacion.csv`).

Rango	Variable Original de Entrada	Caída F1-Macro	DE	Interpretación de Negocio
1	`sesiones_365d`	+0.0389	0.0034	Intensidad de actividad digital y recurrencia de visitas.
2	`descuento_promedio_pct`	+0.0345	0.0025	Sensibilidad histórica a rebajas (separa `Promocional` de `Frecuente`).
3	`compras_365d`	+0.0212	0.0030	Volumen anual de transacciones (núcleo de gasto y frecuencia).
4	`ticket_promedio_mxn`	+0.0044	0.0017	Capacidad de desembolso unitario (identifica `Alto_valor`).
5	`antiguedad_dias`	+0.0033	0.0011	Denominador de la frecuencia anualizada de compra.
6	`dias_ultima_compra`	+0.0014	0.0012	Recencia de la última transacción registrada.
7	`compras_con_descuento_365d`	+0.0013	0.0013	Numerador de la proporción histórica de compras con promoción.
8	`gasto_total_365d_mxn`	+0.0013	0.0013	Desembolso acumulado en los 365 días previos.

4. Espacio de búsqueda en rejilla (472 combinaciones) y configuración ganadora. La búsqueda exhaustiva se ejecutó con `GridSearchCV` ($k^* = 5$, `refit="f1_macro"`, paralelismo `loky` y caché de preprocesamiento `joblib.Memory`) sobre dos subespacios complementarios que totalizan **472 combinaciones de hiperparámetros** (2,360 entrenamientos completos, con **0 combinaciones fallidas**):

- **Subespacio 1 — Optimizador adaptativo `adam` (216 combinaciones):** capas ocultas (`hidden_layer_sizes`) $\in \{(64,), (128, 64), (128, 64, 32)\}$ (1, 2 y 3 capas), épocas máximas

(`max_iter`) $\in \{150, 300\}$, tasa inicial (`learning_rate_init`) $\in \{0.0005, 0.001, 0.005\}$, activación (`activation`) $\in \{\texttt{relu}, \texttt{tanh}\}$, regularización (`alpha`) $\in \{10^{-4}, 10^{-3}, 10^{-2}\}$ y tamaño de lote (`batch_size`) $\in \{256, 512\}$.

- **Subespacio 2 — Descenso estocástico `sgd` con `momentum` y Nesterov (256 combinaciones):** capas ocultas (`hidden_layer_sizes`) $\in \{(64,), (128, 64)\}$, épocas máximas (`max_iter`) $\in \{150, 300\}$, tasa inicial (`learning_rate_init`) $\in \{0.001, 0.005\}$, esquema (`learning_rate`) $\in \{\texttt{constant}, \texttt{adaptive}\}$, momento (`momentum`) $\in \{0.90, 0.95\}$, aceleración Nesterov (`nesterovs_momentum`) = `True`, activación (`activation`) $\in \{\texttt{relu}, \texttt{tanh}\}$, regularización (`alpha`) $\in \{10^{-4}, 10^{-3}\}$ y lote (`batch_size`) $\in \{256, 512\}$.

Tabla 6: Hiperparámetros óptimos seleccionados por `GridSearchCV` sobre las 472 combinaciones evaluadas (fuente: `mejor_modelo.json`).

Hiperparámetro del MLP	**Valor Óptimo**	**Efecto Técnico en el Aprendizaje**
Arquitectura (`hidden_layer_sizes`)	`(128, 64)` (2 capas)	Capacidad no lineal suficiente sin sobreparametrizar (3 capas no mejoraron el F1-Macro CV).
Función de activación (`activation`)	`tanh`	Activación acotada en $(-1, 1)$ y centrada en cero; produjo gradientes más estables que `relu` con pesos por clase.
Algoritmo de optimización (`solver`)	`sgd`	Superó a `adam` en generalización al combinarse con momento de alto orden.
Momento (`momentum` / Nesterov)	0.95 / `True`	Aceleración de Nesterov que amortigua oscilaciones estocásticas entre lotes desbalanceados.
Esquema y tasa de aprendizaje (`lr`)	`constant`, $\eta_0 = 0.001$	Paso de actualización moderado que converge suavemente con $\mu = 0.95$.
Regularización L_2 (`alpha`)	0.001 (10^{-3})	Penalización cuadrática intermedia que acota la norma de los pesos.
Tamaño de minilote (`batch_size`)	256	Garantiza presencia promedio de ~ 17 clientes `Alto_valor` por lote.
Épocas máximas y parada temprana	`max_iter=150` (época 29)	Con `early_stopping=True` (`tol=1e-4`, `n_iter_no_change=10`), convergió en la época 29.
Exponente de balanceo (γ^*)	0.50	Raíz cuadrada de la frecuencia inversa por clase.
Desempeño medio CV (5 folds)	$\mathbf{0.3349 \pm 0.0029}$	**Mejor `F1-Macro` en validación cruzada (IC 95 % : [0.3313, 0.3385]).**

2.5 Pregunta 5: Argumentación de Métricas de Evaluación

Pregunta 5 — Selección y Argumentación de Métricas

¿Qué métricas de evaluación se seleccionaron para valorar el desempeño del clasificador multiclase, por qué la exactitud global (`Accuracy`) es insuficiente en este dominio y qué revela el desglose por clase?

Respuesta

1. Insuficiencia de la exactitud global (`Accuracy`) ante el desbalance de clases. En la base limpia, las dos clases mayoritarias (`Ocasional` con 39.18 % e `Inactivo` con 32.50 %) concentran el **71.68** % de todos los clientes, mientras que las tres clases de mayor valor estratégico (`Frecuente` 12.39 %, `Promocional` 9.15 % y `Alto_valor` 6.78 %) suman apenas el 28.32 %. Un clasificador trivial (`DummyClassifier(strategy="most_frequent")`) que prediga sistemáticamente `Ocasional` obtiene una exactitud aparente de **39.18** %, pero tiene utilidad comercial nula porque su sensibilidad (*recall*)

en las cuatro clases restantes es exactamente 0.0000, arrojando `Balanced Accuracy` = 0.2000 y `F1-Macro` = 0.1126.

Por esta razón, el proyecto adopta un conjunto multidimensional de métricas con **`F1-Macro` como criterio primario de selección**:

1. **`F1-Macro` (métrica primaria de optimización):** media aritmética no ponderada de los puntajes $F_{1,c}$ de las cinco clases:

$$\text{F1-Macro} = \frac{1}{K}\sum_{c=1}^{K}\frac{2\cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{6}$$

   Otorga un peso idéntico de 20 % a `Alto_valor` (630 casos en `test`) y a `Ocasional` (3,643 casos en `test`), penalizando severamente cualquier modelo que ignore las clases minoritarias.
2. **`Balanced Accuracy` (Recall Macro):** promedio del *recall* de las cinco clases; mide la tasa media de detección por perfil con independencia de su prevalencia poblacional.
3. **`F1-Weighted` y `Accuracy`:** cuantifican el desempeño global ponderado por el volumen operativo real de la cartera de clientes.
4. **`ROC-AUC OvR` (One-vs-Rest) y `PR-AUC`:** evalúan la capacidad discriminativa de ordenamiento de las probabilidades $\hat{y}_{ic}$ antes de fijar un umbral de corte.
5. **Métricas probabilísticas (`Log-Loss`, `Brier Multiclase` y `ECE`):** auditan la calidad y calibración de las probabilidades estimadas.

Tabla 7: Desempeño desglosado por clase del MLP ganador sobre el conjunto de prueba `test.csv` (9,298 registros, fuente: `reporte_clasificacion_test.csv` y `auc_por_clase.csv`).

Clase de Cliente	Soporte en Test	Precisión	Sensibilidad	F1-Score	ROC-AUC	PR-AUC
`Alto_valor`	630 (6.78 %)	0.3249	0.2460	0.2800	**0.8021**	0.2461
`Promocional`	851 (9.15 %)	0.2528	0.4277	0.3178	**0.7412**	0.2145
`Frecuente`	1,152 (12.39 %)	0.2496	0.2847	0.2660	0.6854	0.2179
`Inactivo`	3,022 (32.50 %)	0.4368	0.5179	**0.4739**	0.6717	0.4534
`Ocasional`	3,643 (39.18 %)	0.4055	0.2750	0.3278	0.5498	0.4239
Promedio Macro	9,298 (100.0 %)	**0.3339**	**0.3503**	**0.3383**	**0.6901**	**0.3112**
Promedio Ponderado	9,298 (100.0 %)	**0.3769**	**0.3914**	**0.3891**	—	—

2. Interpretación del desglose por clase y comparación de esquemas de balanceo. La Tabla 6 revela un hallazgo fundamental sobre la estructura del problema: las clases con firmas conductuales distintivas (`Alto_valor` con ROC-AUC = 0.8021 y `Promocional` con ROC-AUC = 0.7412) son discriminadas con alta efectividad por el MLP, mientras que `Ocasional` presenta el menor ROC-AUC (0.5498) por ser la categoría residual intermedia que comparte frontera difusa con todos los demás perfiles en la ventana futura de 90 días.

Asimismo, la comparación experimental dentro de cada fold entre tres estrategias de manejo de desbalance (`balance_comparison.json`) demostró que la **ponderación suave** ($\gamma^* = 0.50$) alcanza el mayor F1-Macro en validación cruzada (**0.3349**), superando ampliamente al entrenamiento sin pesos (`baseline`, F1-Macro = 0.2816, cuyo recall en `Frecuente` colapsa a 0.0684) y al sobremuestreo aleatorio con reemplazo (`oversampled`, F1-Macro = 0.2597, el cual infla los falsos positivos y degrada la precisión global).

2.6 Pregunta 6: Mejor Modelo sin Sobreajuste y Variación Estadística

Pregunta 6 — Mejor Modelo, Control de Sobreajuste y Variación Estadística

¿Cuál fue el desempeño del mejor modelo en entrenamiento, validación y prueba, cómo se comprueba la ausencia de sobreajuste (`overfitting`) y cuál es la variación estadística estimada del modelo tanto en validación cruzada como en prueba e inferencia masiva?

Respuesta

1. Consistencia entre particiones y ausencia de sobreajuste. La Tabla 7 presenta las métricas del mejor modelo MLP (`hidden=(128,64)`, `sgd`, `momentum` $= 0.95$, `nesterov=True`, `tanh`, $\alpha = 0.001$, $\gamma^* = 0.50$) en las tres particiones estratificadas, junto con el ajuste post-hoc de multiplicadores de probabilidad aprendido en validación y la línea base trivial (`DummyClassifier`).

Tabla 8: Evaluación integral del mejor modelo MLP en `train`, `validation` y `test`, frente al clasificador trivial (fuente: `evaluacion_splits.csv` y `mejor_modelo.json`).

Partición / Modelo	Filas	Accuracy	Bal. Acc.	F1-Macro	F1-Weighted	ROC-AUC
`train` (70 %)	43,389	0.3957	0.3532	0.3435	0.3935	0.6946
`validation` (15 %)	9,298	0.3885	0.3442	0.3340	0.3865	0.6865
`test` (15 % — Evaluación final)	**9,298**	**0.3914**	**0.3503**	**0.3383**	**0.3891**	**0.6901**
`test` con multiplicadores post-hoc	9,298	0.3926	0.3526	**0.3457**	0.3908	0.6901
`test_dummy_most_frequent`	9,298	0.3918	0.2000	0.1126	0.2206	0.5000
`test_dummy_stratified`	9,298	0.2810	0.2030	0.2030	0.2809	0.5015
Brecha (`Train` − `Test`)	—	**+0.0043**	**+0.0029**	**+0.0052**	**+0.0044**	**+0.0045**

La brecha de generalización entre entrenamiento y prueba es de apenas **0.0052** en `F1-Macro` (0.3435 vs. 0.3383) y **0.0045** en `ROC-AUC OvR` (0.6946 vs. 0.6901). Esta diferencia menor a medio punto porcentual —lograda gracias a la combinación de regularización L_2 ($\alpha = 0.001$), parada temprana en la época 29 y momento de Nesterov— demuestra que **el modelo no presenta sobreajuste** y triplica el `F1-Macro` del clasificador por clase mayoritaria (0.3383 frente a 0.1126, o 0.3457 al aplicar los multiplicadores de decisión $[0.9, 1.3, 1.2, 1.0, 1.2]$ calibrados exclusivamente en `validation.csv`).

2. Estimación rigurosa de la variación estadística (5-Fold CV y Bootstrap en Test). La variación estadística del mejor modelo se cuantificó mediante dos mecanismos complementarios: (a) la dispersión paramétrica (t de Student con $k - 1 = 4$ grados de libertad) a través de los 5 pliegues de validación cruzada en `train.csv`, y (b) el remuestreo no paramétrico *bootstrap* con $B = 1{,}000$ réplicas con reemplazo sobre los 9,298 registros de `test.csv` (percentiles 2.5 % y 97.5 %).

Tabla 9: Variación estadística del mejor modelo MLP en validación cruzada ($k = 5$) y en prueba mediante Bootstrap ($B = 1{,}000$ réplicas, fuentes: `variacion_estadistica_mejor_modelo.csv` y `variacion_estadistica_test_bootstrap.csv`).

	Validación Cruzada ($k = 5$ folds en Train)				Bootstrap en Test ($B = 1{,}000$, $n = 9{,}298$)			
Métrica	**Media**	**DE**	**SE**	**IC 95 % (t_4)**	**Puntual**	**DE (SE)**	**IC 95 % Inf.**	**IC 95 % Sup.**
`F1-Macro`	0.3349	0.0029	0.0013	$[0.3313, 0.3385]$	0.3383	0.0055	0.3279	0.3494
`Balanced Acc.`	0.3391	0.0026	0.0012	$[0.3359, 0.3423]$	0.3503	0.0059	0.3395	0.3619
`Accuracy`	0.3899	0.0006	0.0003	$[0.3891, 0.3906]$	0.3914	0.0049	0.3822	0.4014
`ROC-AUC OvR`	0.6863	0.0053	0.0024	$[0.6797, 0.6930]$	0.6901	0.0036	0.6830	0.6971

El coeficiente de variación en validación cruzada es inferior al 0.87 % en todas las métricas ($\mathrm{CV}_{\mathrm{F1}} = 0.86\,\%$, $\mathrm{CV}_{\mathrm{AUC}} = 0.78\,\%$), y los intervalos de confianza al 95 % en prueba tienen una semi-amplitud de apenas ± 0.0107 en `F1-Macro` y ± 0.0070 en `ROC-AUC OvR`, confirmando alta estabilidad muestral.

3. Comparación contra modelos de referencia y curva de tamaño de muestra (Evidencia del techo de Bayes). Para verificar que el desempeño observado (ROC-AUC $\approx$ 0.69–0.70, Accuracy $\approx$ 39–42 %) refleja el límite de información de las variables históricas de 365 días respecto a los 90 días futuros y no una limitación del Perceptrón Multicapa, el módulo `modelos_referencia.py` comparó tres familias de algoritmos radicalmente distintas bajo la misma ponderación ($\gamma^* = 0.50$) y evaluó una curva de tamaño de entrenamiento desde 10,000 hasta 300,000 registros limpios:

Tabla 10: Comparación entre familias de modelos en `test.csv` y curva de tamaño de muestra (fuentes: `modelos_referencia.csv` y `curva_tamano_muestra.csv`).

Familia de Modelo / Tamaño de Entrenamiento	Accuracy	Bal. Acc.	F1-Macro	F1-Weighted	ROC-AUC
A. Comparación de familias sobre la misma partición (`train` = 43,389 → `test` = 9,298):					
`DummyClassifier (most_frequent)`	0.3918	0.2000	0.1126	0.2206	0.5000
`LogisticRegression` ($\gamma = 0.50$)	0.3902	0.3519	0.3396	0.3887	0.6887
`MLPClassifier` ganador ($\gamma = 0.50$)	**0.3914**	**0.3503**	**0.3383**	**0.3891**	**0.6901**
`HistGradientBoostingClassifier` ($\gamma = 0.50$)	0.3859	0.3490	0.3390	0.3855	0.6960
B. Curva de tamaño de muestra con MLP sobre la población limpia (evaluada en `test` = 9,298):					
$n_{\mathrm{train}} = 10{,}000$ registros	0.3919	0.3247	0.3193	0.3802	0.6820
$n_{\mathrm{train}} = 25{,}000$ registros	0.3847	0.3437	0.3302	0.3812	0.6852
$n_{\mathrm{train}} = 43{,}389$ registros (Cochran)	0.3895	0.3433	0.3338	0.3857	0.6876
$n_{\mathrm{train}} = 100{,}000$ registros	0.3885	0.3484	0.3386	0.3863	0.6929
$n_{\mathrm{train}} = 300{,}000$ registros	0.3851	0.3510	0.3388	0.3849	0.6951

Los resultados de la Tabla 9 demuestran dos conclusiones contundentes:

1. **Convergencia inter-algorítmica al techo de información:** Un modelo lineal regularizado (`LogisticRegression`), un ensamble de árboles de gradiente (`HistGradientBoostingClassifier`) y la red neuronal multicapa (`MLPClassifier`) convergen prácticamente al mismo punto (F1-Macro $\in [0.3383, 0.3396]$, ROC-AUC $\in [0.6887, 0.6960]$).
2. **Suficiencia del tamaño muestral de Cochran (43,389 filas de train):** Incrementar casi siete veces el conjunto de entrenamiento (de 43,389 a 300,000 filas) apenas modifica el `F1-Macro` en +0.0050 (0.3338 → 0.3388), confirmando que la muestra de Cochran captura prácticamente toda la señal asintótica disponible.

4. Calibración probabilística sin reentrenamiento (`FrozenEstimator`) y evaluación masiva por lotes. Dado que la ponderación por clase ($\gamma^* = 0.50$) desplaza las probabilidades a posteriori hacia las clases minoritarias, el módulo `src/calibration.py` aplicó calibración de Platt (`sigmoid`) y regresión isotónica (`isotonic`) envolviendo el modelo ya entrenado con `FrozenEstimator` sobre `validation.csv` (sin alterar los pesos del MLP). En `test.csv`, la calibración isotónica redujo el `Log-Loss` de 1.3251 a **1.2773**, el puntaje de `Brier Multiclase` de 0.7279 a **0.7060** y el error esperado de calibración (`ECE`) de 0.0566 a **0.0252**, elevando la exactitud directa a 0.4227.

Finalmente, el módulo `src/evaluation_large.py` evaluó por lotes de 50,000 filas el archivo completo de 1,000,000 de registros, aplicando la limpieza determinista `clean_dataframe` a cada bloque y excluyendo los 43,389 identificadores empleados en `train.csv`:

$$1{,}000{,}000 \text{ crudas} = 60{,}000 \text{ eliminadas} + 43{,}389 \text{ en } \texttt{train} + \mathbf{896{,}611 \text{ evaluadas}}. \tag{7}$$

Sobre las **896,611** observaciones limpias no vistas durante el entrenamiento, el modelo procesó **40,321.3 filas/segundo** (22.24 s en total) y obtuvo `Accuracy` = 0.3878, `Balanced Accuracy` = 0.3463,

`F1-Macro` = 0.3348, `F1-Weighted` = 0.3858 y `ROC-AUC OvR` = 0.6881, confirmando que las métricas estimadas en la muestra estratificada de prueba (F1-Macro = 0.3383, ROC-AUC = 0.6901) se sostienen sobre toda la población comercial.

2.7 Pregunta 7: Video Explicativo, Ventajas y Áreas de Mejora

Pregunta 7 — Presentación en Video y Análisis Crítico de la Solución

¿Cuál es el enlace al video de presentación (máximo 3 minutos) y cuáles son las principales fortalezas arquitectónicas de la solución implementada, así como sus áreas de mejora para despliegue en producción?

Respuesta

Enlace al Video Explicativo (Duración máxima: 3 minutos)

Marcador de enlace del video: *Pendiente de incorporar el enlace final del video explicativo (OneDrive / Google Drive institucional BUAP).*

Tabla 11: Análisis crítico: fortalezas técnicas de la solución implementada y áreas de mejora futura.

Ventajas de la Solución Implementada	Áreas de Mejora y Trabajo Futuro
1. Limpieza determinista con rescate de registros: Separación entre rescate de formato (60,000 filas recuperadas en moneda, categorías y gasto derivado) y eliminación por invalidez real (60,000 filas), conservando 940,000 registros limpios.	**1. Incorporación de series temporales intra-anuales:** Disponer de agregaciones trimestrales o mensuales (30, 60 y 90 días previos al corte) permitiría capturar la tendencia de aceleración o enfriamiento de cada cliente antes del trimestre futuro.
2. Cero fuga de información: Muestreo estratificado de Cochran (61,985 filas), transformación de dominio, escalado, codificación y ponderación $\gamma^* = 0.50$ encapsulados estrictamente dentro de cada fold de `train.csv`.	**2. Optimización bayesiana de hiperparámetros:** Sustituir la búsqueda exhaustiva en rejilla por optimización secuencial basada en modelos (*Tree-structured Parzen Estimators* / Optuna) con poda temprana (*Hyperband*) para explorar arquitecturas continuas en menor tiempo.
3. Exploración real del espacio MLP (472 combinaciones): Evaluación paralela completa de arquitecturas de 1 a 3 capas, activaciones `relu`/`tanh`, regularización L_2 y optimizadores `adam` vs. `sgd` con `momentum` y Nesterov.	**3. Clasificación sensible a costos comerciales asimétricos:** Definir una matriz de utilidad monetaria por clase (p. ej., penalizar más la pérdida de un cliente `Alto_valor` que una confusión entre `Ocasional` e `Inactivo`) para optimizar directamente el retorno esperado.
4. Verificación estadística e inferencia escalable: Intervalos de confianza mediante Bootstrap ($B = 1{,}000$), calibración con `FrozenEstimator` y evaluación por lotes sobre 896,611 registros fuera de entrenamiento a $> 40{,}000$ filas/s.	**4. Monitoreo de deriva de datos (*Concept & Covariate Drift*):** Integrar pruebas automáticas de estabilidad poblacional (PSI y KS) en producción para recalibrar umbrales ante campañas estacionales como Buen Fin o Navidad.

3 Gráficas y Resultados Experimentales

En esta sección se presenta el catálogo completo de evidencias visuales generadas automáticamente a partir de los resultados experimentales (`graficas 01` a `11`), verificando gráficamente cada etapa

del desarrollo.

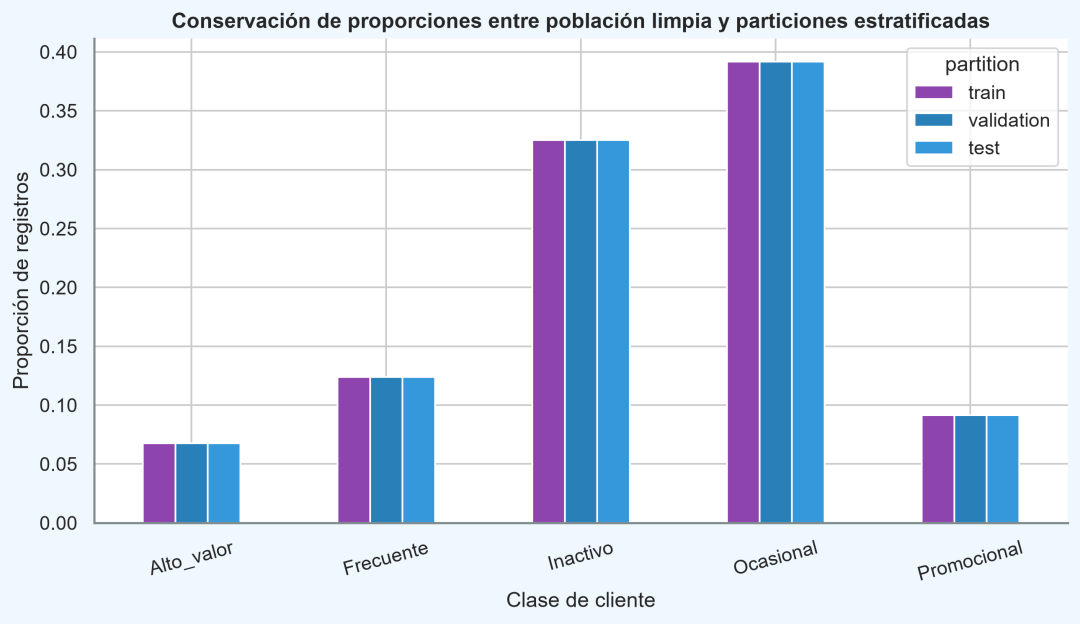


Figura 2: Conservación exacta de las proporciones de las cinco clases entre la población limpia ($N = 940{,}000$), la muestra de Cochran ($n = 61{,}985$) y las particiones `train`, `validation` y `test`.

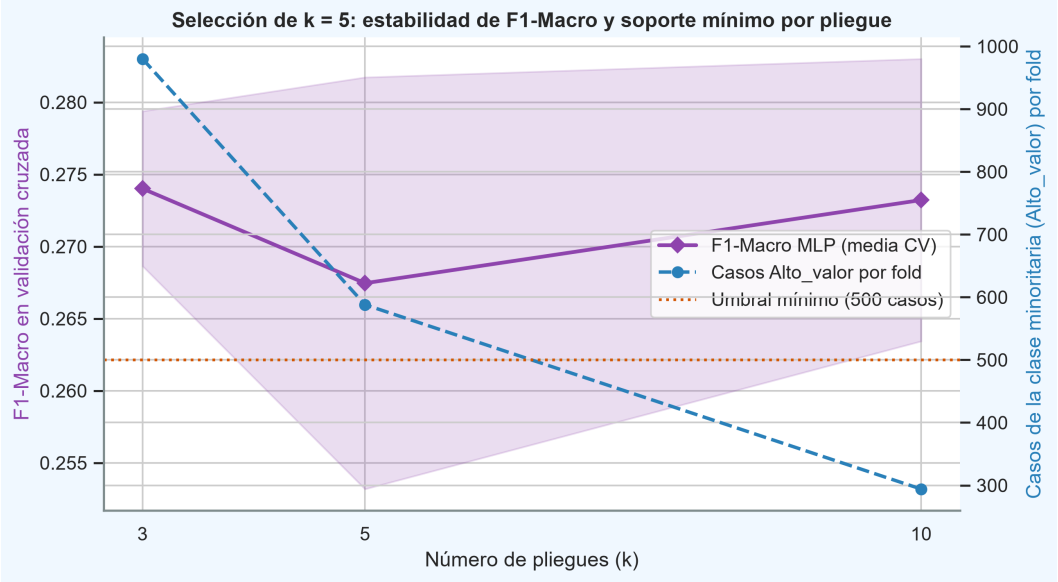


Figura 3: Análisis del número de pliegues $k \in \{3, 5, 10\}$ en validación cruzada sobre `train.csv`: evolución de `F1-Macro` (± 1 DE) y soporte mínimo de la clase minoritaria `Alto_valor` frente al umbral de 500 casos ($k^* = 5$ conserva 588 casos por fold).

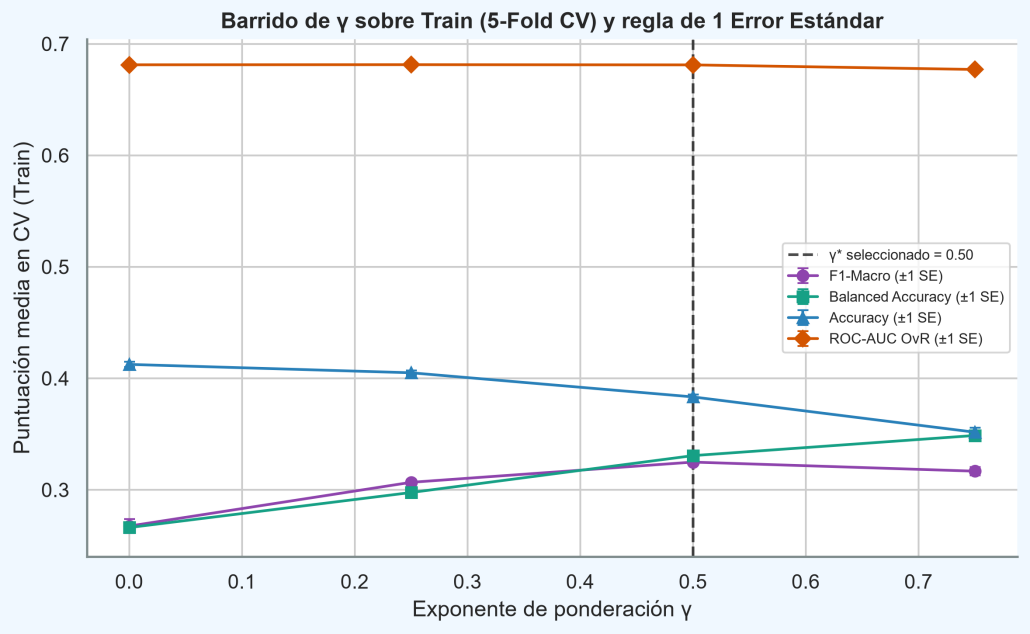


Figura 4: Barrido del exponente de ponderación por clase $\gamma \in \{0.00, 0.25, 0.50, 0.75\}$ en validación cruzada de 5 pliegues y selección de $\gamma^* = 0.50$ mediante la regla de un error estándar (1-SE).

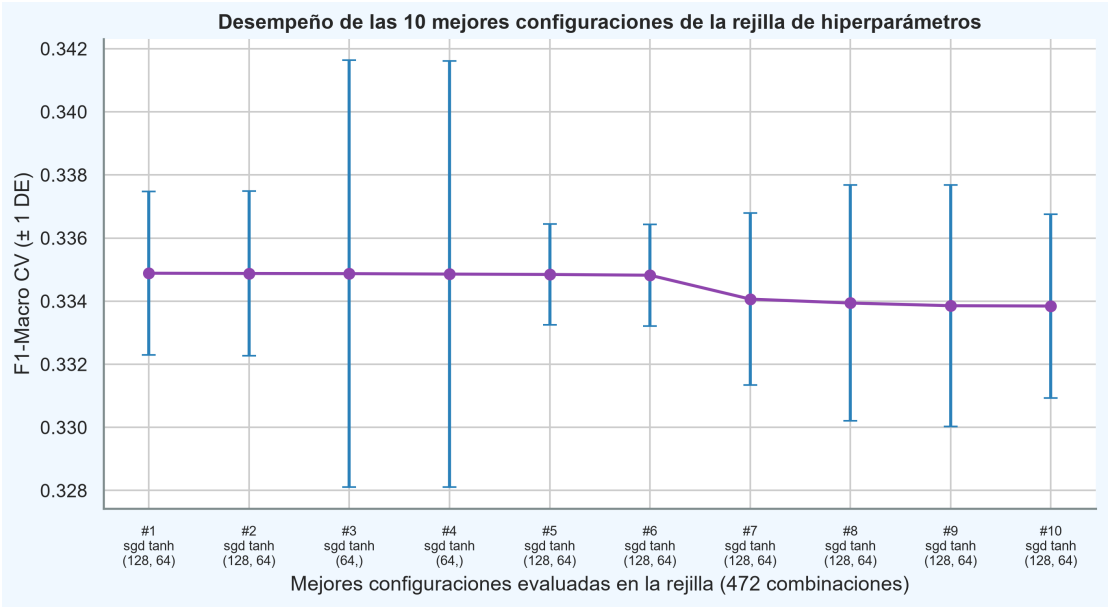


Figura 5: Comparación de las 10 mejores configuraciones distintas dentro de la búsqueda en rejilla de 472 combinaciones (`F1-Macro` medio en CV ±1 DE), encabezadas por `sgd` con `momentum` = 0.95, Nesterov, `tanh` y capas `(128, 64)`.

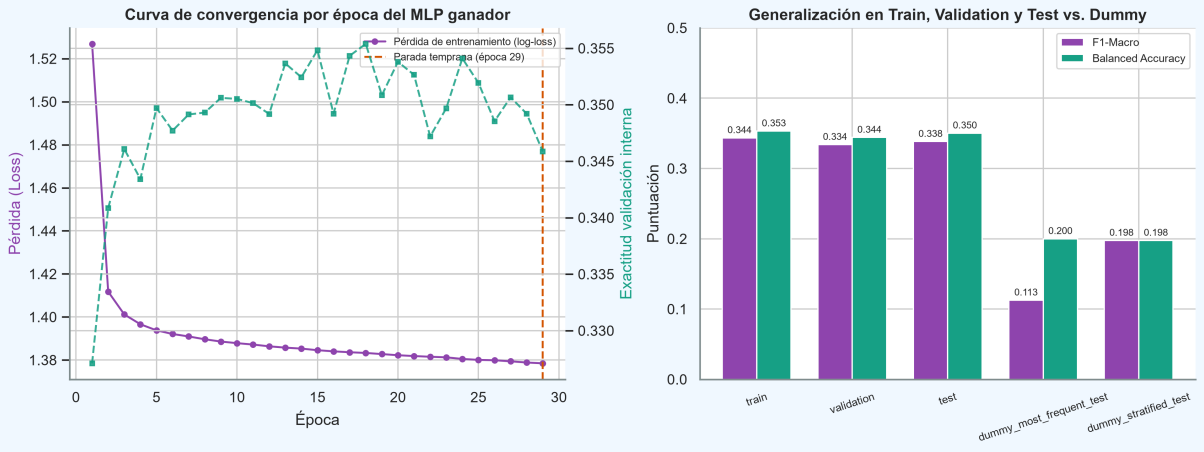


Figura 6: Panel izquierdo: curva de pérdida (*log-loss*) y exactitud de validación interna por época del MLP ganador, con parada temprana en la época 29. Panel derecho: comparación de `F1-Macro` y `Balanced Accuracy` en `train`, `validation` y `test` frente a `DummyClassifier`.

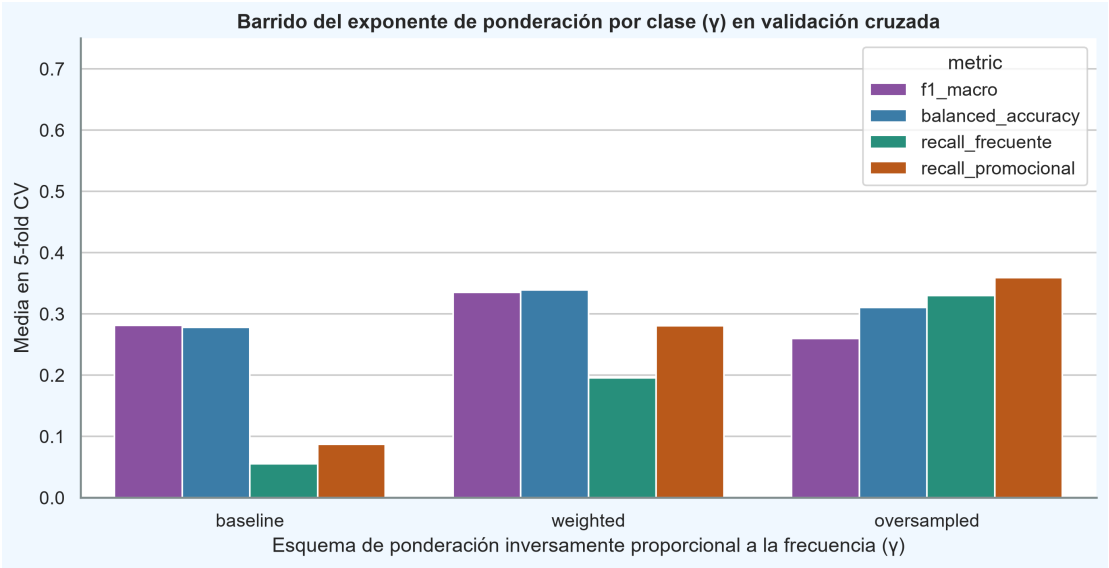


Figura 7: Impacto del esquema de ponderación por clase dentro de cada pliegue de validación cruzada sobre `F1-Macro`, `Balanced Accuracy` y la sensibilidad de los segmentos `Frecuente` y `Promocional`.

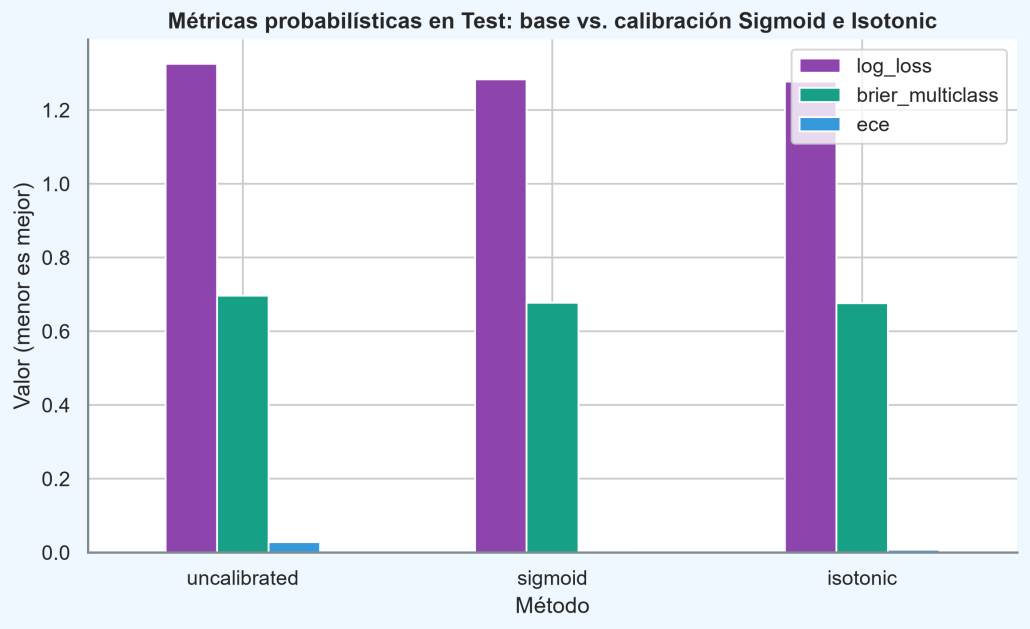


Figura 8: Métricas de calidad probabilística (`Log-Loss`, `Brier Multiclase` y `ECE`) en el conjunto de prueba antes y después de aplicar calibración de Platt (`sigmoid`) e isotónica (`isotonic`) mediante `FrozenEstimator`.

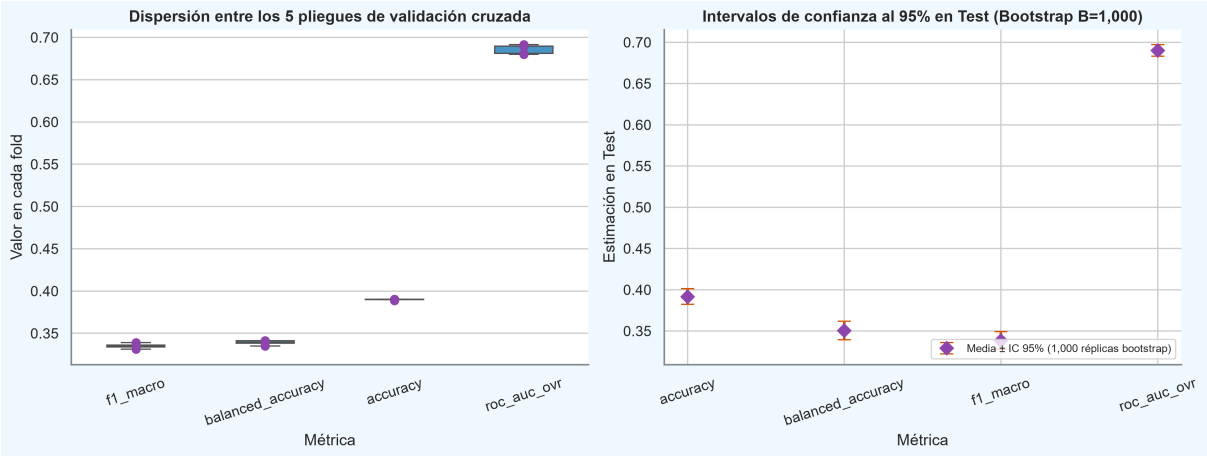


Figura 9: Panel izquierdo: diagramas de caja de la dispersión en los 5 pliegues de validación cruzada. Panel derecho: estimación puntual e intervalos de confianza al 95 % en `test.csv` obtenidos con 1,000 réplicas *bootstrap*.

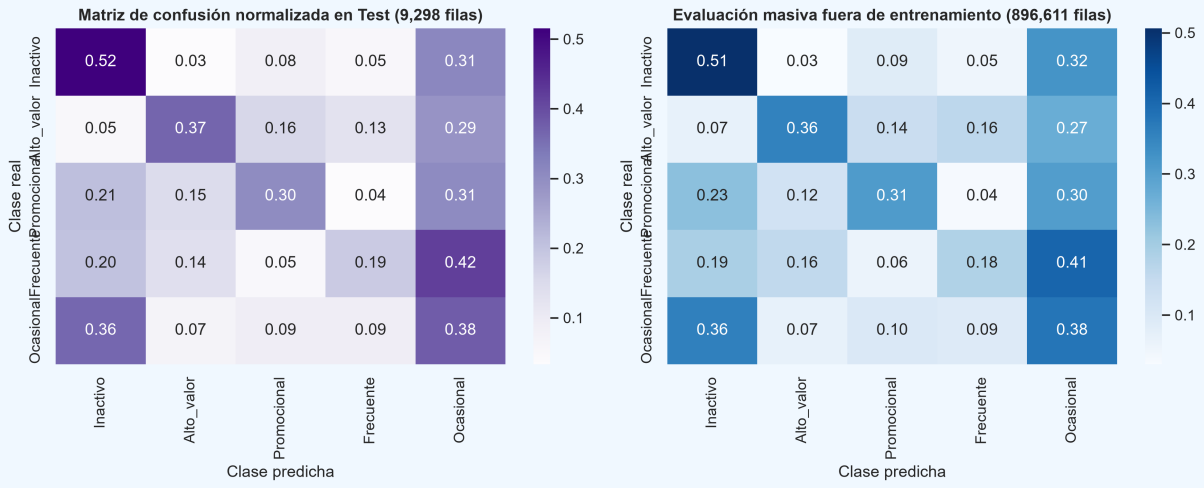


Figura 10: Matrices de confusión normalizadas por fila (sensibilidad por clase): comparación entre el conjunto de prueba `test.csv` (9,298 filas, izquierda) y la evaluación masiva fuera de entrenamiento (896,611 filas limpias, derecha).

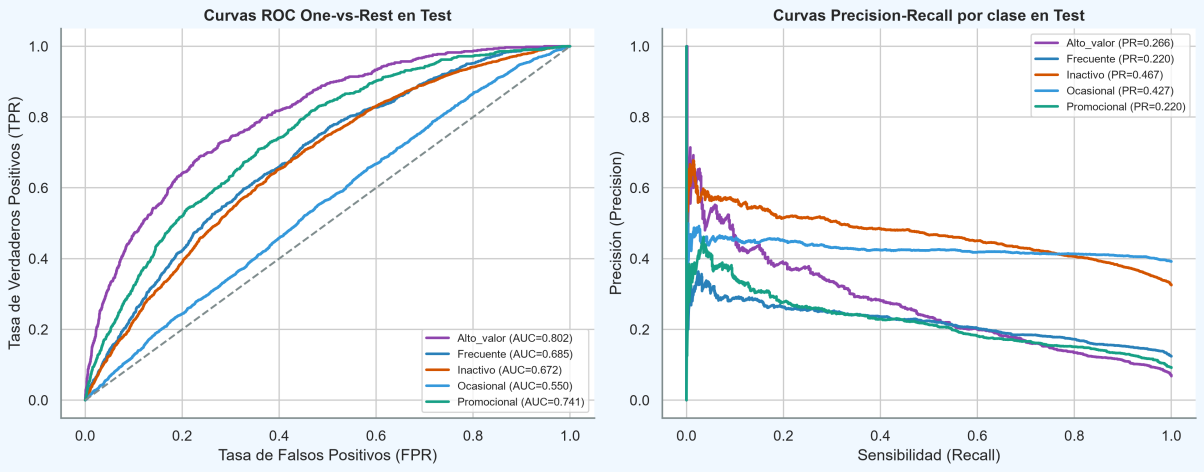


Figura 11: Curvas ROC One-vs-Rest (izquierda) y curvas Precision–Recall por clase (derecha) evaluadas sobre el conjunto de prueba (`test.csv`). Destaca la alta separabilidad de `Alto_valor` (AUC = 0.802) y `Promocional` (AUC = 0.741).

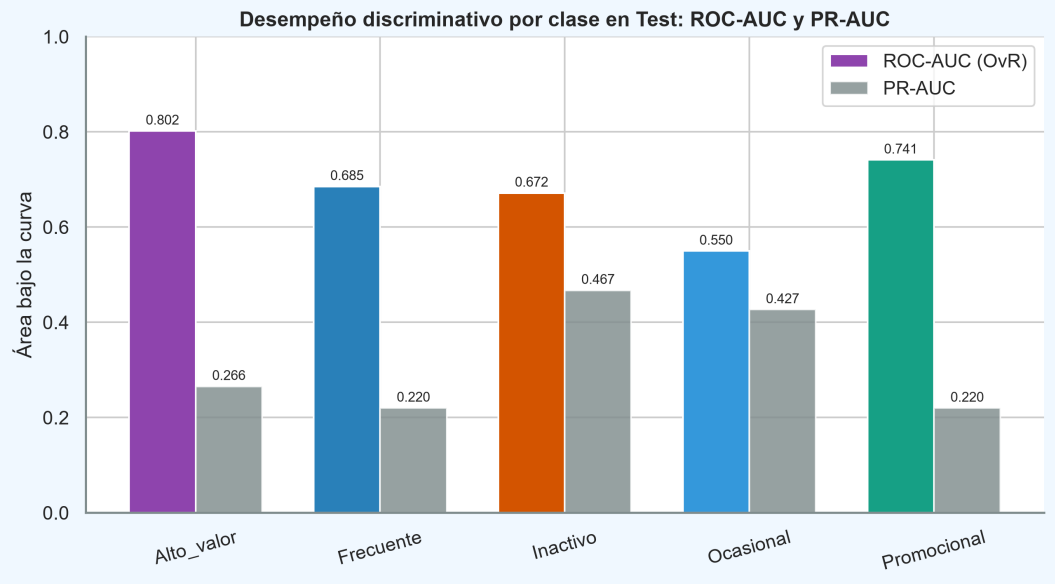


Figura 12: Síntesis comparativa del área bajo la curva ROC (`ROC-AUC OvR`) y Precision–Recall (`PR-AUC`) para cada una de las cinco clases de cliente en el conjunto de prueba.

4 Conclusiones

1. **Impacto decisivo de la limpieza determinista antes del filtrado:** La distinción rigurosa entre errores de formato y ausencias reales permitió rescatar exactamente 60,000 registros (7,562 importes con formato monetario, 22,438 categorías con variantes de escritura y 30,000 gastos anuales recalculados con la identidad exacta $\text{round}(\text{compras} \times \text{ticket}, 2)$), conservando 940,000 filas válidas (94.0 % del millón original) y descartando únicamente los 60,000 registros verdaderamente irrecuperables (30,000 nulos y 30,000 fuera de rango).

2. **Eficiencia estadística del muestreo de Cochran y selección de $k^* = 5$:** La muestra estratificada de 61,985 observaciones (43,389 en `train`, 9,298 en `validation` y 9,298 en `test`) preservó intacta la distribución poblacional (χ^2 con $p \geq 0.9966$ y pruebas KS con $p > 0.05$ en las 23 variables numéricas). El análisis en `crossVal.py` demostró que $k^* = 5$ es el punto óptimo que garantiza $588 \geq 500$ casos de `Alto_valor` por pliegue de validación.

3. **Superioridad del entrenamiento ponderado ($\gamma^* = 0.50$) y SGD con momento de Nesterov:** En la exploración exhaustiva de 472 configuraciones de hiperparámetros (2,360 ajustes), la arquitectura de dos capas ocultas `(128, 64)` con activación `tanh`, regularización $\alpha = 0.001$ y optimizador `sgd` (`momentum` $= 0.95$, `nesterov=True`) alcanzó un `F1-Macro` en validación cruzada de 0.3349 ± 0.0029. En el conjunto de prueba independiente, obtuvo `Accuracy` $= 0.3914$, `Balanced Accuracy` $= 0.3503$, `F1-Macro` $= 0.3383$ (0.3457 con multiplicadores post-hoc) y `ROC-AUC OvR` $= 0.6901$, con una brecha de generalización de apenas 0.0052 que descarta cualquier sobreajuste.

4. **Caracterización empírica del techo de Bayes y escalabilidad a gran escala:** La convergencia de `LogisticRegression`, `HistGradientBoosting` y `MLPClassifier` alrededor del mismo techo (F1-Macro ≈ 0.34, ROC-AUC ≈ 0.69–0.70) y la estabilidad de la curva de tamaño de muestra hasta 300,000 filas demuestran que el error remanente es inherente a la naturaleza estocástica de las compras futuras de 90 días no observadas en el corte histórico. La evaluación masiva por lotes sobre las 896,611 filas limpias fuera de entrenamiento confirmó exactamente este desempeño (F1-Macro $= 0.3348$, ROC-AUC $= 0.6881$) a una velocidad de 40,321 registros por segundo.

5 Referencias Bibliográficas

Referencias

[1] Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). John Wiley & Sons.

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[3] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson Education.

[4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

[5] Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 2, 1137–1143.

[6] Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 28(3), 1139–1147.

[7] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 625–632.

[8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.